

Syllabus Details

Syllabus ID:	14178
Syllabus Name:	Software Testing_Kiểm thử phần mềm
Course Name English:	Software Testing
Subject Code:	SWT301
NoCredit:	3
Degree Level:	Bachelor
Time Allocation:	Study hour (150h) = 45h (60 sessions) contact hours + 145-minute final exam + 102,6h self-study
Pre-Requisite:	SWE102 or SWE201c or SWE202c
Description:	• General principles of software testing, including its purposes and objectives. • Testing integrated into software development processes, covering test levels, test types, and main • Static testing basics, including feedback and review processes to enhance software quality. • Techniques such as black-box (functional), white-box (structural), and experience-based testing a • Test activities like planning, estimation, monitoring, risk management, defect reporting, and test au • Tools and methods to support effective testing, emphasizing benefits and potential risks. • AI is used to explore testing concepts, support static testing, and create/optimize Test Plans.
StudentTasks:	Students must attend at least 80% of contact sessions in order to be accepted to the final examinatio - Students are responsible to do all exercises, assignments and labs given by instructor in class or at - The use of AI in assessments and exams must comply with the instructions of the course instructor - Use laptop in class only for learning purpose - Promptly access to the https://flm.fpt.edu.vn/ for up-to-date course information
Tools:	- Unit Test Framework - Bug Management Tool - Ms Office - IDE - Static Analyst Tool - Automation Tool - AI Tools: Grok (xAI), ChatGPT, Gemini, Deepseek
Scoring Scale:	10
DecisionNo MM/dd/yyyy:	377/QĐ-ĐHFPT dated 04/09/2026
IsApproved:	True
Note:	In the case: (5 > Final TE Score >=4) & (5 > Final PE Score >=4) & FR < 5, the student can choose to
MinAvgMarkToPass:	5
IsActive:	True
ApprovedDate:	4/9/2026

4 material(s)

MaterialDescription
Foundations of Software Testing: ISTQB Certification
Agile Testing: A Practical Guide for Testers and Agile Teams
Computer Software Validation. Alan Muirhead. News & Views, March 2000. Available at http://www.stcpmc.org/archive/archive
Course Software Testing and Quality Assurance by Tian in http://engr.smu.edu/~tian/class/7314.06f/syl.html (availble in FU libr

10 LO(s)

CLO Name	CLO Details
CLO1	Know definitions, concepts and terminologies about Software Testing, using AI to explore testing principles and re
CLO10	Demonstrate presentation skills and effective teamwork
CLO2	Understand the software testing process. Distinguish between levels of testing (e.g. unit, integration, system, acce
CLO3	Use static testing techniques to detect errors through a review process, using AI to review documents for defects (
CLO4	Use dynamic testing techniques like black-box, white-box, and experience-based testing to design and execute te
CLO5	Develop a comprehensive test plan for an AI-enabled project, clearly defining its scope, goals, resources, and test
CLO6	Analyze and prioritize risks to focus testing efforts effectively. Understand and apply bug logging, bug reporting, a
CLO7	Classify and select appropriate testing tools, evaluate the benefits and risks of test automation, and choose an aut
CLO8	Understand and apply Agile testing methods.
CLO9	Utilize AI tools to support learning and exercises related to the course.

[View mapping of CLOs to PLOs](#)

Download All Teacher Material

Download All Student Material

Session	Topic	Learning-T
1	- Course Introduction - Introducing AI integration in teaching and learning, as well as assessment	Offline
2	Topic 1. Fundamentals of testing 1.1 What is testing?	Offline
3	1.2 Why is testing necessary?	Offline
4	1.3 Testing principles	Offline
5	1.4 Fundamental test process	Offline
6	1.5 The psychology of testing 1.6 Codes of ethics	Offline
7	Guiding Lab 1 (Use chat AI to explore testing fundamentals through effective prompts)	Offline
8	Topic 2. Testing throughout the software life cycle 2.1 Software development models	Offline
9	2.2 Test levels	Offline
10	2.2 Test levels (cont.)	Offline

11	2.3 Test types: the targets of testing	Offline
12	2.4 Maintenance testing	Offline
13	Progress test 1	Offline
14	Review Lab 1	Offline
15	Guiding Lab 2 (Install and study an unit test framework, Use IDE to write unit test code and run unit test)	Offline
16	Topic 3. Static techniques 3.1 Reviews and the test process	Offline
17	Topic 3. Static techniques 3.2 Review process	Offline
18	3.2 Review process (cont.)	Offline
19	3.2 Review process (cont.)	Offline
20	3.3 Static analysis by tools	Offline
21	Guiding Lab 3 (Static techniques: Manually review documents or use AI for static code analysis)	Offline
22	Topic 4. Test design techniques 4.1 The test development process	Offline
23	4.2 Categories of test design techniques	Offline
24	4.3 Specification-based or black-box techniques	Offline
25	4.3 Specification-based or black-box techniques (cont.)	Offline
26	4.3 Specification-based or black-box techniques (cont.)	Offline
27	4.3 Specification-based or black-box techniques (cont.)	Offline

28	4.4 Structure-based or white-box techniques	Offline
29	4.5 Experience-based techniques	Offline
30	4.6 Choosing a test technique	Offline
31	Progress test 2	Offline
32	Review Lab 2	Offline
33	Review Lab 3	Offline
34	Topic 5. Test management 5.1 Test organization	Offline
35	5.2 Test plans, estimates, and strategies	Offline
36	Guiding Lab 4 (Create Test Plan with AI)	Offline
37	5.3 Test progress monitoring and control	Offline
38	5.4 Configuration management	Offline
39	5.5 Risk and testing	Offline
40	5.6 Incident management	Offline
41	Review Lab 4	Offline
42	Review Problems	Offline
43	Topic 6. Tool support for testing 6.1 Types of test tool	Offline
44	6.2 Effective use of tools: Potential benefits and risks	Offline

45	6.3 Introducing a tool into an organization	Offline
46	Progress Test 3	Offline
47	Topic 7. ISTQB Foundation Exam 7.1 Preparing for the exam 7.2 Taking the exam	Offline
48	7.3 Mock exam	Offline
49	Topic 8. Agile Testing 8.1 The fundamentals of Agile software development 8.2 Aspects of Agile approaches	Offline
50	8.3 Mostly used Agile Methods 8.4 Fundamental Agile Testing Principles, Practicles and Processes	Offline
51	8.5 Agile Testing Methods, Techniques and Toools	Offline
52	Individual presentation and mark	Offline
53	Individual presentation and mark	Offline
54	Individual presentation and mark	Offline
55	Individual presentation and mark	Offline
56	Individual presentation and mark	Offline
57	Individual presentation and mark	Offline
58	Individual presentation and mark	Offline
59	Individual presentation and mark	Offline
60	Individual presentation and mark	Offline

45 Constructive question(s)

	Session No	Name	Details

1	1	CQ1.1	What challenges might arise when integrating AI into teaching software testing, and how can we address them?
2	2	CQ1.2	How would you explain the concept of "testing" to someone who has no background in software development?
3	3	CQ1.3	How does testing contribute to building trust in software systems for end-users?
4	4	CQ2.1	Why do you think defect clustering is a common phenomenon in software projects?
5	5	CQ2.2	How would you adapt the fundamental test process for a small project with limited resources?
6	6	CQ2.3	How can a tester's mindset differ from a developer's mindset, and why does this matter in testing?
7	7	CQ3.1	Why should ethical considerations, such as avoiding bias, be a priority in software testing?
8	8	CQ3.2	What are the advantages of integrating testing early in a V-Model compared to a Waterfall model?
9	9	CQ3.3	How does writing unit tests in an IDE help you understand the code's functionality?
10	10	CQ4.1	Why might integration testing reveal defects that were missed during component testing?
11	11	CQ4.2	How would you justify the need for acceptance testing to a project manager who wants to skip it?
12	12	CQ4.3	How would you decide whether to prioritize functional or non-functional testing for a mobile app?
13	13	CQ5.1	Why is regression testing critical after a software update, and how can it be optimized?
14	14	CQ5.2	How does maintenance testing differ from testing during initial development?
15	15	CQ5.3	What factors would you consider when planning maintenance testing for a legacy system?
16	16	CQ6.1	Why are reviews considered an effective way to find defects early in the development process?
17	17	CQ6.2	How does the role of a moderator impact the success of a review session?
18	18	CQ6.3	How would you handle a situation where team members disagree on a defect found during a review?
19	19	CQ7.1	Why is follow-up after a review as important as the review itself?
20	20	CQ7.2	What limitations of static analysis tools should you be aware of when using them?
21	21	CQ7.3	What criteria would you use to decide whether to use AI or manual methods for static analysis?
22	22	CQ8.1	Why is it important to define test conditions before designing test cases?
23	23	CQ8.2	When might you choose an experience-based technique over a specification-based one?
24	24	CQ8.3	How does equivalence partitioning reduce the number of test cases while maintaining coverage?
25	25	CQ9.1	Why is boundary value analysis particularly effective for finding defects?
26	26	CQ9.2	How can you use decision tables to ensure all possible scenarios are tested?
27	27	CQ9.3	How does state transition testing help in testing systems with complex workflows?
28	28	CQ10.1	Why might you combine multiple black-box techniques in a single test effort?
29	29	CQ10.2	Why might decision coverage be more valuable than statement coverage in critical systems?
30	30	CQ10.3	How can exploratory testing uncover defects that structured techniques might miss?
31	31	CQ11.1	What skills are most important for a tester using experience-based techniques?
32	32	CQ11.2	How would you decide which test technique to use for a time-constrained project?
33	33	CQ11.3	Why might the choice of test technique vary depending on the project's risk level?
34	34	CQ12.1	How does a clear test organization structure improve the efficiency of a testing team?
35	35	CQ12.2	Why is accurate test estimation critical to project success?
36	36	CQ12.3	How can metrics help you monitor testing progress effectively?
37	37	CQ13.1	What actions would you take if testing progress falls behind schedule?
38	38	CQ13.2	Why is configuration management essential for maintaining test integrity?
39	39	CQ13.3	How can risk analysis guide your testing efforts in a high-stakes project?
40	40	CQ14.1	What steps would you take to mitigate a newly identified risk during testing?

41	41	CQ14.2	How would you prioritize incidents found during testing to ensure efficient resolution?
42	42	CQ14.3	Why is it important to document incidents thoroughly even if they are not fixed immediately?
43	43	CQ15.1	How do different types of test tools support various stages of the testing process?
44	44	CQ15.2	What risks might arise from over-relying on testing tools, and how can you mitigate them?
45	45	CQ15.3	Why is a pilot project important when introducing a new testing tool to an organization?

6 assessment(s)

Category	Type	Part	Weight	Completion Criteria	Duration
Lab	on-going	4	25.0%	>0	Option 1: Lab 1: 2-6 weeks; Lab 2-4: 2 weeks. Option 2 (For Constructivism Approach only): Follow lecturer
Presentation	on-going	1	10.0%	>0	Option 1: 10'/each; Option 2 (For Constructivism Approach or
Progress Test	on-going	3	15.0%	>0	Option 1: 30'/each; Option 2 (For Constructivism Approach o
Final exam	Final exam	2	50.0%	4	145 minutes
Practical Exam	Final exam	1	25.0%	4	85'
Theory Exam	Final exam	1	25.0%	4	60'